

CSCE 650 VIRTUAL REALITY · MODULE 04 · LECTURE NOTES

Display Optics and Distortion

What the lens does, and what you must undo.

MODULE OUTCOMES

- Explain why HMDs need lenses at all.
- Explain barrel and pincushion distortion and the correction order.
- Explain chromatic aberration correction.
- Explain why distortion must happen after timewarp.
- Name the display artefacts and their causes.

1 Why there is a lens

THE EYE CANNOT FOCUS ON A DISPLAY 3 CM AWAY

The near point of accommodation — the closest distance an eye can bring into focus — is roughly 10 cm for a young adult and recedes with age. Focusing near that limit is effortful and uncomfortable well before reaching it.

A headset display sits 3 to 5 cm from the eye. Without optics it would be an unfocusable blur.

The lens forms a virtual image at 1.5 to 2 m, where the eye is relaxed and can comfortably remain for long periods. **It also magnifies**, so a panel a few centimetres across subtends a field of view around 100° — which is the other reason it is there, and arguably the more important one.

And it introduces distortion, chromatic aberration, a fixed focal plane, and several named artefacts.

The lens creates roughly as many problems as it solves, and this module is about the ones that fall to software.

	What the lens does	What you do about it
Distortion	Magnification increases with distance from the optical axis, so straight lines bow inward. This is pincushion distortion, and it is worse at wider fields of view.	Pre-distort the submitted image with the inverse — barrel distortion. The two cancel and the user sees straight lines. This is why a raw VR frame looks visibly warped when viewed on a monitor, which is how most people first encounter the subject.

	What the lens does	What you do about it
Chromatic aberration	Different wavelengths refract by different amounts, so red, green, and blue are magnified slightly differently. Uncorrected, high-contrast edges near the periphery acquire coloured fringes.	Apply a slightly different distortion mesh per colour channel. The correction is never exact, which is why fringing remains faintly visible at the extreme edges of most headsets if you look for it deliberately.

2 Where distortion happens, and why the order matters

```

APPLICATION                                COMPOSITOR (the runtime)
1. predict head pose
2. render left eye ----->
3. render right eye ----->
   (flat, undistorted,
    linear images)
4. take the LATEST head pose
   (newer than step 1's)
5. TIMEWARP to that pose
6. apply LENS DISTORTION (per channel)
7. scan out

```

DISTORTION MUST COME AFTER TIMEWARP.

WHY DISTORTION CANNOT BE APPLIED BY THE APPLICATION

Timewarp operates in undistorted image space. It is, in essence, a rotation of the rendered image to account for how the head has moved since the frame was rendered (Module 06). That rotation is only meaningful on an image with a straightforward projective relationship to the world.

Applying timewarp to an already-distorted image warps the distortion itself. The barrel correction is no longer the exact inverse of the lens's pincushion, so the cancellation fails and the user sees residual warping that varies with head motion — which is both visible and nauseating.

So the ordering is forced: the application submits flat, undistorted images; the compositor reprojects them to the newest available pose, and only then applies the lens correction.

This is the answer to 'why can I not do the distortion myself', which is a reasonable question with a specific answer.

- **The distortion mesh is device-specific and sometimes unit-specific,** derived from factory calibration of the individual lenses. You do not have the data and should not want it.
- **It must compose correctly with timewarp,** which the compositor applies after your frame is submitted.
- **It varies with eye relief and IPD,** because the required correction depends on where the eye actually sits relative to the lens.
- **Submit flat undistorted images and let the runtime handle it.** The first generation of VR SDKs required applications to perform their own distortion, and the resulting inconsistency is part of why OpenXR exists.

3 What distortion does to your pixels

THE PERIPHERY IS COMPRESSED AND THE CENTRE IS STRETCHED

Barrel pre-distortion maps the submitted image onto the display non-uniformly: **the centre is stretched outward and the edges are compressed inward.**

So peripheral pixels you rendered are discarded. Several submitted pixels near the edge collapse into a single displayed pixel — all the shading work that produced them is thrown away.

And central pixels are magnified. One submitted pixel near the centre covers more than one displayed pixel, so if the submitted image matched the panel resolution the centre of the view — where the user is looking — would be soft.

This is why the runtime's recommended render target exceeds the panel resolution, frequently by 30 to 40% in each dimension. You are deliberately rendering surplus pixels so that the centre survives the warp at full sharpness, and accepting that the periphery's surplus is wasted.

- **Radial density masking** skips rendering the extreme corners of the submitted image, which fall outside the visible lens area entirely and are never displayed under any circumstances.
- **The runtime supplies a hidden-area stencil mesh** marking exactly those pixels. Rendering with it bound costs nothing and saves a meaningful fraction of the fragment work. **It is the single easiest performance win in VR and it is routinely left unapplied,** usually because nobody knew it existed.
- **Fixed foveated rendering** (Module 10) reduces the shading rate toward the edges of the image — which the distortion is about to compress anyway, so much of the detail would have been discarded regardless.
- **The distortion's compression and the eye's acuity falloff (Module 02) agree.** Both say that peripheral detail is worth less, which is convenient: one optimisation satisfies two independent arguments.

Artefact	Cause	What to do
Screen door effect	Visible dark gaps between pixels — a consequence of fill factor and of magnifying the panel.	Nothing in software. It diminishes with panel density, and modern headsets have largely overcome it.
Mura	Per-pixel variation in brightness and colour arising from panel manufacturing, magnified along with everything else.	Corrected by a calibration map in the runtime. Visible on uniform dark fields, which is one reason to avoid large flat dark areas.
God rays	Internal reflection between the concentric steps of a Fresnel lens , producing radial streaks from bright objects.	Avoid bright objects on dark backgrounds — white text on black is the worst case. This is a genuine content constraint on Fresnel-lens headsets.
Chromatic fringing	Imperfect aberration correction, worst at the field edges.	Nothing; it is residual error in the runtime's correction.
Pupil swim	The distortion correction assumes a fixed eye position; the eye rotates.	See below.

Artefact	Cause	What to do
Glare and ghosting	Reflections from the lens surfaces.	Reduce extreme contrast; the rest is hardware.

PUPIL SWIM, AND THE CASE FOR EYE TRACKING

The distortion correction is calibrated for the eye in one position — looking straight ahead at a nominal eye relief.

When the eye rotates to look at something off-centre, the pupil moves, and the geometry the correction assumed no longer holds. The correction is then slightly wrong, in a way that changes as the gaze changes.

The world appears to warp subtly as you look around. It is faint, most people cannot describe it, and it is a genuine contributor to the impression that a virtual space is not quite solid — one of those effects that is below the threshold of articulation and above the threshold of perception.

Eye tracking permits dynamic distortion correction, adjusting the mesh for the current pupil position. **This is one of the less-discussed benefits of eye tracking and arguably matters more than foveated rendering**, because it improves the experience rather than merely the frame rate.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford EE267 — optics and distortion lectures and assignment (free)	Builds the distortion correction from the lens equation and has you implement it. The best free treatment of this module.
LaValle — Virtual Reality, Chapter 4 (free)	Light and optics, including the aberrations and why Fresnel lenses behave as they do.
Oculus — asynchronous timewarp and the compositor (free developer blog archive)	Why the compositor owns distortion, written when the decision was being made.
OpenXR — visibility mask extension (free)	The hidden-area mesh of §3. Short specification, immediate benefit.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a screenshot of a submitted VR frame before distortion and view it on a monitor. Describe the barrel warping.

- 2 Implement barrel distortion as a post-process on a flat image and verify that applying pincushion afterwards recovers the original.
- 3 Implement per-channel chromatic correction on a test image with high-contrast edges, and measure the residual fringing at the corners.
- 4 Deliberately apply distortion before a simulated timewarp rotation and show that the correction no longer cancels.
- 5 Compare the runtime's recommended render target size against the headset's panel resolution and compute the ratio.
- 6 **Apply the hidden-area stencil mesh** and measure the fragment work saved. Report the percentage.
- 7 Render white text on black and look for god rays. Then render the same text dark-on-light and compare.
- 8 Look at a uniform dark grey field in a headset and look for mura.
- 9 Look at the extreme edge of the field while keeping your head still, and then rotate only your eyes. Try to detect pupil swim.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does a headset need a lens? Give two reasons.
- 2 What distortion does the lens produce, and what do you render to cancel it?
- 3 How is chromatic aberration corrected, and why is it imperfect?
- 4 Why must distortion be applied after timewarp?
- 5 Give three reasons not to implement your own distortion.
- 6 What does pre-distortion do to peripheral and central pixels, and what follows for render target size?
- 7 What is the hidden-area mesh and why should you always use it?
- 8 Name five display artefacts and their causes.
- 9 What is pupil swim, and what fixes it?